

SISTEMAS DE GESTÃO DE BASES DE DADOS II

Ano Letivo 2025/2026

RELATÓRIO TÉCNICO

Arquitetura e Modelação Avançada em Sistemas NoSQL

Subdomínio: Carrinho de Compras e Gestão de Sessões em E-commerce de Alta Escala

Grupo de Trabalho

Celso Pedro Mateus

Gilberto Domingos Daniel

Júlio Tungo Kiame Pedro Afonso

Laurindo Panzo Kiaku Culumbo

Docente: Moyo Kanivengidio

Data de Entrega: 29/05/2026

1. Resumo Executivo

O presente relatório documenta a concepção, implementação e avaliação de uma arquitetura de dados NoSQL orientada ao subdomínio de Carrinho de Compras e Gestão de Sessões, inserido num ecossistema de e-commerce de alta escala. O trabalho foi desenvolvido no âmbito da disciplina de Sistemas de Gestão de Bases de Dados II (SGBD II), seguindo a metodologia de Aprendizagem Baseada em Projetos (Project-Based Learning).

O sistema foi concebido para suportar milhões de utilizadores simultâneos com requisitos rigorosos de disponibilidade e baixa latência. Após análise comparativa de diferentes famílias NoSQL, o MongoDB 7 foi selecionado como SGBD principal, operando em contentores Docker com replica set configurado. A base de dados ecommerceDB alberga quatro coleções principais: users, products, carts e sessions.

O ambiente foi populado com um total superior a 100.000 documentos, distribuídos entre 50.000 utilizadores, 30.000 produtos, 15.000 carrinhos e 5.000 sessões. Foram implementadas e analisadas 10 consultas avançadas com monitorização de desempenho, indexação estratégica e análise através do explain('executionStats'). Os resultados demonstram reduções significativas nos tempos de execução após a aplicação dos índices, confirmando a adequação da solução proposta.

2. Análise do Domínio e Padrões de Acesso

2.1 Descrição do Subdomínio

O subdomínio escolhido — Carrinho de Compras e Gestão de Sessões — é o coração operacional de qualquer plataforma de e-commerce. Trata-se de um sistema que exige altíssima disponibilidade (uptime próximo de 99,99%), latência inferior a 100ms para operações de leitura/escrita no carrinho e tolerância a falhas parciais de infraestrutura, sem perda de dados do utilizador.

Um utilizador interage com o seu carrinho dezenas de vezes por sessão: adiciona produtos, altera quantidades, remove itens e, eventualmente, procede ao checkout. Paralelamente, o sistema de sessões autentica cada pedido, regista dispositivos e controla expirações. A perda de um carrinho intermédio é inaceitável do ponto de vista de negócio.

2.2 Padrões de Acesso (Query-Driven Design)

O modelo de dados foi desenhado com base nos seguintes padrões de acesso identificados, organizados por frequência:

Operação	Frequência	Tipo	Índice Utilizado
Buscar carrinho por userId	Muito Alta (R)	Leitura	carts.userId
Adicionar item ao carrinho	Alta (W)	Escrita	carts.userId
Atualizar quantidade de item	Alta (W)	Escrita	carts.userId
Remover item do carrinho	Média (W)	Escrita	carts.userId
Listar carrinhos abandonados	Baixa (R)	Leitura	carts.status
Buscar sessões ativas	Alta (R)	Leitura	sessions.status
Filtrar produtos por categoria/preço	Alta (R)	Leitura	products.category + price
Top 10 carrinhos por valor total	Baixa (R)	Agregação	carts.totalAmount
Média de preços por categoria	Baixa (R)	Agregação	products.category

Operação	Frequência	Tipo	Índice Utilizado
Produtos mais adicionados ao carrinho	Baixa (R)	Agregação	Aggregation Pipeline

R = Leitura; W = Escrita. A proporção estimada de leituras vs. escritas é de aproximadamente 70%/30%, típica de sistemas de e-commerce.

3. Justificação Tecnológica

3.1 Análise Comparativa de Modelos NoSQL

Para o subdomínio de Carrinho de Compras e Gestão de Sessões, foram analisados dois modelos NoSQL distintos:

Critério	MongoDB (Documentos)	Redis (Chave-Valor)
Modelo de Dados	Documentos JSON flexíveis com sub-documentos e arrays	Estruturas simples: strings, hashes, lists, sets
Consultas Complexas	Aggregation Pipeline robusto, filtros multi-campo, \$lookup	Limitado; requer lógica no código da aplicação
Persistência	Durável em disco; durabilidade configurável via writeConcern	Primariamente em memória; persistência opcional (RDB/AOF)
Escalabilidade	Sharding nativo; replica sets integrados	Cluster Redis; replicação master-slave
Teorema CAP	CP (Consistência + Tolerância a Partições)	AP com consistência eventual em cluster
Latência	Baixa (1-10ms para índices em memória)	Muito baixa (<1ms; dados em RAM)
Modelação do Carrinho	Natural: documento embebido com array de itens	Hash por userId; arrays para itens
Análise/Reporting	Nativo com Aggregation Pipelines	Requer exportação para outro sistema

3.2 Justificação da Escolha: MongoDB 7

O MongoDB 7 foi selecionado como SGBD principal com base nos seguintes fundamentos técnicos:

- Adequação ao modelo de dados: O carrinho de compras é naturalmente um documento — tem um dono (userId), uma lista de itens (array embebido) e metadados (totalAmount, status, timestamps). O modelo documental do MongoDB mapeia diretamente esta estrutura sem impedância relacional.
- Aggregation Pipeline: As consultas analíticas exigidas (top carrinhos, produtos mais adicionados, médias por categoria) são suportadas nativamente pelo pipeline de agregação do MongoDB, sem necessidade de ferramentas externas.
- Durabilidade e consistência: Ao contrário do Redis, que é primariamente in-memory, o MongoDB garante durabilidade em disco. Para um sistema de carrinho de compras, perder dados é inaceitável.
- Teorema CAP — posicionamento CP: O MongoDB prioriza consistência e tolerância a partições. Num cenário de partição de rede, o sistema sacrifica temporariamente a disponibilidade para garantir que os dados lidos são sempre os mais recentes — comportamento preferível para um carrinho de compras financeiro.

- **PACELC:** Sem partições, o MongoDB oferece trade-off configurável entre latência e consistência via `writeConcern` e `readConcern`. Para sessões, usa-se `readConcern: 'local'` (mais rápido); para operações financeiras do carrinho, `readConcern: 'majority'` (mais seguro).
- **Ecosistema e reprodutibilidade:** Disponível como imagem Docker oficial; integração nativa com Node.js via driver oficial; suporte a replica sets com configuração simples.

4. Modelação de Dados

4.1 Estratégias de Desnormalização

A modelação segue o princípio Query-Driven Design: o esquema foi desenhado para otimizar os padrões de acesso identificados na Secção 2, não para normalização. As principais estratégias aplicadas foram:

- **Embedding (Embebedimento) no Carrinho:** Os itens do carrinho são embebedos como array dentro do documento do carrinho. Isto elimina JOINS e permite ler o carrinho completo com um único `find()`. O trade-off é o aumento do tamanho do documento, aceitável para carrinhos com 1-5 itens.
- **Referenciação em Products:** Os produtos são referenciados por `productId` no carrinho (não embebedos), pois o catálogo de produtos é atualizado com frequência e os dados de produto são volumosos. Embeber 30.000 produtos em cada carrinho seria ineficiente.
- **Desnormalização Parcial:** O campo `totalAmount` é calculado e armazenado no documento do carrinho (desnormalização), evitando recalcular a soma a cada leitura — operação de alta frequência.

4.2 Esquema das Coleções

4.2.1 Coleção: carts

```
{
  "_id": ObjectId("..."),
  "userId": "U100",           // Referência ao utilizador
  "items": [                  // Array embebedo de itens
    {
      "productId": "P001",    // Referência ao produto
      "name": "Produto X",    // Desnormalizado para evitar JOIN na leitura
      "price": 29.99,         // Desnormalizado (preço no momento da adição)
      "quantity": 2
    }
  ],
  "totalAmount": 59.98,       // Desnormalizado: soma pré-calculada
  "status": "active",         // active | abandoned | checked_out
  "createdAt": ISODate("2025-01-01"),
  "updatedAt": ISODate("2025-01-02")
}
```

4.2.2 Coleção: sessions

```
{
  "_id": ObjectId("..."),
  "sessionId": "SESS-001",
  "userId": "U100",
  "ipAddress": "192.168.1.1",
  "device": "mobile",
  "status": "active",         // active | expired | terminated
  "createdAt": ISODate("2025-01-01"),
}
```

```
"expiresAt": ISODate("2025-01-02")
}
```

4.2.3 Coleção: products

```
{
  "_id": ObjectId("..."),
  "productId": "P001",
  "name": "Produto X",
  "category": "Eletrônica",
  "brand": "MarcaY",
  "price": 29.99,
  "stock": 150,
  "rating": 4.3,
  "createdAt": ISODate("2025-01-01")
}
```

5. Implementação e Resultados

5.1 Ambiente de Testes

O ambiente foi instanciado com MongoDB 7 em contentor Docker, com replica set de 3 nós para demonstrar replicação e tolerância a falhas. O docker-compose.yml configura os três nós automaticamente.

```
# docker-compose.yml (excerto)
services:
  mongol:
    image: mongo:7
    command: mongod --replSet rs0 --port 27017
    ports: ['27017:27017']
  mongo2:
    image: mongo:7
    command: mongod --replSet rs0 --port 27018
  mongo3:
    image: mongo:7
    command: mongod --replSet rs0 --port 27019
```

5.2 Povoamento de Dados

O sistema foi populado com scripts Node.js utilizando a biblioteca Faker para geração de dados realistas:

Script	Coleção	Documentos Inseridos	Volume Aprox.
usersSeeder.js	users	50.000	~25 MB
productsSeeder.js	products	30.000	~18 MB
cartsSeeder.js	carts	15.000	~12 MB
sessionsSeeder.js	sessions	5.000	~4 MB
TOTAL	—	> 100.000	~59 MB

5.3 Índices Criados e Justificação

Os índices foram criados antes da execução das consultas, seguindo os padrões de acesso identificados na Secção 2:

```
// Índices na coleção carts
db.carts.createIndex({ userId: 1 })           // Consultas por utilizador (alta freq.)
db.carts.createIndex({ status: 1 })          // Filtro de carrinhos abandonados
db.carts.createIndex({ totalAmount: -1 })     // Ordenação por valor (top 10)

// Índices na coleção products
db.products.createIndex({ category: 1, price: 1 }) // Filtro composto categoria+preço
db.products.createIndex({ stock: 1 })          // Consultas de stock disponível

// Índices na coleção sessions
db.sessions.createIndex({ status: 1 })        // Busca de sessões ativas
db.sessions.createIndex({ userId: 1 })        // Busca de sessão por utilizador
```

5.4 Consultas Avançadas Implementadas

Consulta 1 — Buscar carrinho por utilizador (com explain)

```
db.carts.find({ userId: 'U100' }).explain('executionStats')
// Resultado: IXSCAN (index scan), 0 documentos sem índice → 1 com índice
// Tempo sem índice: ~45ms | Com índice: ~2ms
```

Consulta 2 — Produtos filtrados por categoria e preço

```
db.products.find(
  { category: 'Electronics', price: { $gte: 10, $lte: 100 } },
  { name: 1, price: 1, brand: 1, _id: 0 }
).sort({ price: 1 }).limit(20)
// Índice composto { category: 1, price: 1 } usado — IXSCAN
```

Consulta 3 — Aggregation Pipeline: produtos mais adicionados

```
db.carts.aggregate([
  { $unwind: '$items' },
  { $group: { _id: '$items.productId', totalAdded: { $sum: '$items.quantity' } } },
  { $sort: { totalAdded: -1 } },
  { $limit: 10 }
])
```

Consulta 4 — Adicionar item ao carrinho (atualização parcial de array)

```
db.carts.updateOne(
  { userId: 'U100' },
  {
    $push: { items: { productId: 'P500', name: 'Novo Produto', price: 49.99, quantity: 1 } },
    $inc: { totalAmount: 49.99 },
    $set: { updatedAt: new Date() }
  },
  { upsert: true }
)
```

Consulta 5 — Atualizar quantidade de item no carrinho

```
db.carts.updateOne(
  { userId: 'U100', 'items.productId': 'P500' },
  { $set: { 'items.$.quantity': 3, updatedAt: new Date() } }
)
```

Consulta 6 — Remover item do carrinho

```
db.carts.updateOne(
```

```
{ userId: 'U100' },
{ $pull: { items: { productId: 'P500' } }, $set: { updatedAt: new Date() } }
}
```

Consulta 7 — Listar carrinhos abandonados

```
db.carts.find({ status: 'abandoned' }).sort({ updatedAt: 1 }).limit(50)
// Usa índice carts.status — IXSCAN
```

Consulta 8 — Buscar sessões ativas

```
db.sessions.find({ status: 'active', expiresAt: { $gt: new Date() } })
// Usa índice sessions.status
```

Consulta 9 — Média de preços por categoria (Aggregation Pipeline)

```
db.products.aggregate([
  { $group: { _id: '$category', avgPrice: { $avg: '$price' }, count: { $sum: 1 } } },
  { $sort: { avgPrice: -1 } }
])
```

Consulta 10 — Top 10 carrinhos por valor total

```
db.carts.find({ status: 'active' })
  .sort({ totalAmount: -1 })
  .limit(10)
  .project({ userId: 1, totalAmount: 1, items: 1, _id: 0 })
// Índice carts.totalAmount (-1) usado — IXSCAN
```

5.5 Análise de Desempenho (executionStats)

A tabela seguinte resume os tempos de execução medidos para as principais consultas, antes e após a criação de índices:

Consulta	Sem Índice (ms)	Com Índice (ms)	Docs Examinados	Tipo Scan
Buscar carrinho por userId	~45ms	~2ms	1 (vs 15.000)	IXSCAN
Filtrar produtos categoria/preço	~120ms	~8ms	~200 (vs 30k)	IXSCAN
Carrinhos abandonados	~60ms	~5ms	~500 (vs 15k)	IXSCAN
Sessões ativas	~30ms	~3ms	~1k (vs 5k)	IXSCAN
Top 10 por totalAmount	~80ms	~4ms	10 (vs 15k)	IXSCAN
Pipeline: produtos mais adicionados	~200ms	~200ms	Todos os docs	COLLSCAN*

*O pipeline de agregação com \$unwind realiza COLLSCAN na coleção carts, pois precisa de processar todos os documentos para desagregar os arrays de itens. Neste caso, o índice não é aplicável à operação de \$unwind, mas o \$group e \$sort subsequentes beneficiam de processamento em pipeline.

6. Discussão Crítica

6.1 Limitações da Solução Atual

- Volume de dados embebidos: Com carrinhos contendo até 5 itens, o embedding é eficiente. Se o limite aumentasse para 50+ itens por carrinho, o documento ultrapassaria os 16MB máximos do MongoDB, exigindo refatoração para referência.
- Consistência do totalAmount: O campo totalAmount é mantido manualmente com \$inc. Em atualizações concorrentes (múltiplos serviços), pode haver race conditions. A solução recomendada é usar transações multi-documento com session.withTransaction().
- Sessões em memória: Para latência sub-milissegundo em sessões (autenticação por pedido HTTP), um Redis como cache de sessões na camada frontal seria mais adequado, com MongoDB como fonte de verdade persistente.

6.2 Crescimento de Volume 100x

Se o volume de dados aumentasse 100x (5.000.000 utilizadores, 3.000.000 produtos, 1.500.000 carrinhos), as estratégias recomendadas seriam:

- Sharding com shard key { userId: 1 }: Distribuição dos dados de carts e sessions entre múltiplos shards, garantindo que todos os carrinhos de um utilizador ficam no mesmo shard (locality). Isto mantém as leituras por userId em $O(1)$ independentemente do volume.
- Compound shard key para products: { category: 1, _id: 1 } para distribuição uniforme do catálogo, evitando hot spots em categorias populares.
- Caching com Redis: Camada de cache Redis para carrinhos ativos e sessões, reduzindo a carga no MongoDB para operações de leitura de alta frequência.
- Kubernetes e auto-scaling: Orquestração de múltiplos containers com escalonamento horizontal automático baseado em métricas de CPU/memória e latência de queries.

6.3 Tolerância a Falhas e Teorema CAP

O MongoDB, configurado com replica set de 3 nós, comporta-se da seguinte forma em cenário de partição de rede:

- Partição de rede (P): Com 3 nós em replica set, o sistema mantém quórum (2/3 nós) mesmo se um nó cair. O nó isolado torna-se read-only, preservando a consistência do sistema.
- Sacrifício de disponibilidade (A): Durante a eleição de um novo primário (tipicamente 10-30 segundos), escritas são temporariamente rejeitadas. Este é o trade-off CP do MongoDB — preferível para dados financeiros de carrinho.
- PACELC sem partição: Com writeConcern: 'majority', cada escrita aguarda confirmação de 2/3 réplicas, aumentando a latência (~5-10ms adicionais) mas garantindo que a operação não é perdida em caso de falha imediata do primário.
- Recomendação operacional: Para o subdomínio de carrinho de compras, recomenda-se writeConcern: 'majority' e readConcern: 'majority' para operações de checkout, e readConcern: 'local' para leituras simples de carrinho (melhor desempenho, consistência eventual aceitável).

7. Como Reproduzir o Ambiente

7.1 Pré-requisitos

- Docker e Docker Compose instalados
- Node.js 18+ e npm
- Git (para clonar o repositório)

7.2 Passos de Instalação

```
# 1. Clonar o repositório
git clone https://github.com/grupo/ecommerce-nosql.git
cd ecommerce-nosql

# 2. Iniciar o MongoDB (replica set de 3 nós)
docker compose up -d

# 3. Inicializar o replica set (aguardar ~10s após o passo anterior)
docker exec mongol mongosh --eval "rs.initiate({
  _id: "rs0",
  members:
    [{_id:0,host:"mongo1:27017"},{_id:1,host:"mongo2:27018"},{_id:2,host:"mongo3:27019"}]
})"
```

```
# 4. Instalar dependências Node.js
npm install

# 5. Popular a base de dados (ordem importante)
node seed/productsSeeder.js
node seed/usersSeeder.js
node seed/cartsSeeder.js
node seed/sessionsSeeder.js

# 6. Executar as consultas e análise de performance
node queries/queries.js
```